STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: SE_001_VCS
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: Software Engineering & Agile Framework	STATUS: In Progress

Practice Worksheet – Version Control and Release Workflow

Master professional Git branching strategies, merge conflict resolution protocols, automated semantic release tagging, and production CI/CD deployment pipelines.

KEY LEARNING OBJECTIVES:

- Implement enterprise Git Flow branching models (main, develop, feature, hotfix).
- Resolve complex multi-file merge conflicts while preserving commit history integrity.
- Establish Semantic Versioning (SemVer 2.0.0) release tagging standards.
- Architect automated CI/CD staging-to-production deployment and rollback workflows.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Review the release scenario and define the Git branching taxonomy required for team isolation.
Step 2: Formulate step-by-step CLI commands for branch creation, rebasing, and merge pull requests.
Step 3: Establish automated branch protection rules and peer code review gates.
Step 4: Document the release validation and emergency rollback strategy in the worksheet.

EXERCISE 1.1: GIT FLOW BRANCHING TAXONOMY & PROTECTION RULES

Problem Statement: An enterprise fintech team has 8 developers contributing concurrently. Design a rigorous Git Flow branching strategy detailing the purpose, source, and destination of 'main', 'develop', 'feature/*', 'release/*', and 'hotfix/*' branches. Specify 3 critical branch protection rules for 'main' (e.g., minimum approvals, status checks, linear history).

Reference Baseline / Code / Scenario:

```
git checkout -b feature/auth-flow develop && git push origin feature/auth-flow
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Branching Hierarchy & Rules: _____

[2] Branch Protection Directives: _____

[3] CLI Commands Workflow: _____

EXERCISE 1.2: MULTI-FILE MERGE CONFLICT TRIAGE & REBASE STRATEGY

Problem Statement: Two developers modified 'OrderService.java' and 'application.yml' simultaneously. Developer A re-architected payment gateways while Developer B updated database pooling. Detail the step-by-step interactive rebase protocol (`git rebase -i develop`) to resolve conflicting hunks cleanly without creating an unwanted merge commit.

Reference Baseline / Code / Scenario:

```
CONFLICT (content): Merge conflict in src/main/java/com/ordercraft/OrderService.java
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Conflict Analysis & Hunk Review: _____

[2] Interactive Rebase Protocol: _____

[3] Verification & Fast-Forward Merge: _____

EXERCISE 1.3: SEMANTIC VERSIONING (SEMVER) & RELEASE TAGGING

Problem Statement: Your team is preparing release v2.4.0 with backwards-compatible performance enhancements, two minor bug fixes, and one deprecated internal API. Formulate the SemVer version bump calculation, release tag commands with annotated GPG signatures, and automated GitHub/GitLab release notes generation.

Reference Baseline / Code / Scenario:

```
git tag -a v2.4.0 -m 'Release v2.4.0: Performance upgrades and payment fixes'
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] SemVer Bump Calculation: _____

[2] Signed Tag Commands: _____

[3] Changelog Extraction Strategy: _____

EXERCISE 1.4: CI/CD DEPLOYMENT GATES & INSTANT ROLLBACK PROTOCOL

Problem Statement: Construct a deployment pipeline schema for containerized Spring Boot microservices deploying to Kubernetes. The workflow must define automated linting, unit test execution, staging deployment, canary traffic shifting (10% -> 50% -> 100%), and automatic rollback if error rates exceed 0.5% within 5 minutes.

Reference Baseline / Code / Scenario:

```
pipeline { stages { stage('Test') { steps { sh 'mvn test' } } } }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] CI/CD Pipeline Stages: _____

[2] Canary Deployment Gates: _____

[3] Automated Rollback Trigger: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Branch Isolation:** Feature and hotfix branches isolated with strict lifecycle rules.
- [ ] **Conflict Handling:** Interactive rebase and hunk resolution preserves linear history.
- [ ] **SemVer Standards:** Version increments adhere strictly to Major.Minor.Patch guidelines.
- [ ] **CI/CD Automation:** Automated testing, build gating, and canary releases defined.
- [ ] **Rollback Readiness:** Explicit recovery steps provided for catastrophic production bugs.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is interactive rebasing preferred over repetitive merge commits in enterprise codebases?*
- *What automated guardrail in a CI/CD pipeline is most critical to prevent faulty code from hitting users?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) [ ] NEEDS REVISION (Resubmit Corrected Work) **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Best Practices